

IF2224 - Teori Bahasa Formal dan Otomata

Tugas Besar

Milestone 3 : Semantic Analysis

7 Mei 2026 - 23 Mei 2026



Mirza Tsabita Wafa'ana 13524114

Nathanael Shane Bennet 13524119

Faris Wirakusuma Triawan 13524130

Jonathan Harijadi 13524144

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

DAFTAR ISI

BAB 1 : Teori Singkat.....	3
1.1. Kompilator (Compiler).....	3
1.2. Semantic Analysis.....	3
1.3. Abstract Syntax Tree (AST).....	3
1.5. Attributed Grammar & Visit Functions.....	4
BAB 2 : Perancangan & Implementasi.....	5
2.2. Implementasi Program.....	5
1. File include/SemanticAnalyzer.hpp dan src/SemanticAnalyzer.cpp.....	5
2. File include/SymbolTable.hpp dan src/SymbolTable.cpp.....	5
2.1 Komponen Struktur Data Symbol Table.....	5
2.2 Komponen Logika Semantic Analyzer.....	8
2.3. Aturan Kompatibilitas Tipe (Type Compatibility Rules).....	9
2.4. Aturan Kompatibilitas Penugasan (Assignment Compatibility Rules).....	9
2.5. Definisi Tipe Dasar (Simple Types).....	10
1. Daftar Predefined Identifier.....	10
2. Semantic Actions.....	11
BAB 3 : Pengujian.....	12
BAB 4 : Kesimpulan dan Saran.....	18
4.1. Kesimpulan.....	18
4.2. Saran.....	18
Lampiran.....	19
Referensi.....	20

BAB 1 : Teori Singkat

1.1. Kompilator (*Compiler*)

Kompilator merupakan suatu program yang bertugas menerjemahkan kode sumber (*source code*) yang ditulis dalam bahasa pemrograman tingkat tinggi menjadi kode yang dapat dieksekusi oleh mesin. Proses kompilasi secara umum terbagi menjadi beberapa tahap, yaitu analisis leksikal (*lexical analysis*), analisis sintaksis (*syntax analysis*), analisis semantik (*semantic analysis*), optimasi kode, dan pembangkitan kode target. Tahap pertama dan paling mendasar dari proses kompilasi adalah analisis leksikal, yang menjadi fokus utama pada proyek ini (Aho, Lam, Sethi, & Ullman, 2006).

1.2. Semantic Analysis

Analisis semantik adalah pemeriksaan makna program yang tidak dapat diekspresikan oleh CFG. Analisis semantik diperlukan karena Context-Free Grammar hanya menangani struktur, bukan makna (konteks). Analisis semantik memeriksa berbagai hal, seperti :

- Apakah variabel sudah dideklarasikan sebelum digunakan
- Apakah variabel digunakan sebelum diinisialisasi
- Apakah tipe operand kompatibel satu sama lain
- Apakah jumlah/argumen dan tipe argumen fungsi sesuai
- Mismatch dalam tipe
- dll.

1.3. Abstract Syntax Tree (AST)

Abstract Syntax Tree (AST) adalah bentuk ringkas dari *parse tree* yang hanya menyimpan informasi krusial untuk tahapan kompilasi selanjutnya. Berbeda dengan *parse tree* yang berfokus pada aturan *grammar* untuk pengecekan sintaksis, AST berfokus pada hubungan semantik antar operator dan operand . AST menghilangkan informasi sintaksis yang redundan, seperti tanda kurung berlebih, kata kunci terminal (contoh: **BEGIN**, **END**), dan *semicolon*.

1.4. Decorated Abstract Syntax Tree (Decorated AST)

Decorated AST adalah hasil akhir dari proses analisis semantik di mana AST telah dianotasi dengan informasi tambahan. Anotasi ini bertujuan memberikan konteks semantik yang diperlukan untuk tahap pembangkitan kode, meliputi:

- Referensi atau indeks yang mengarah ke entri yang tepat di *Symbol Table*.
- Tipe data akhir dari suatu ekspresi.
- Informasi tingkatan *lexical level* atau *scope* tempat node tersebut berada.

1.5. Attributed Grammar & Visit Functions

Attributed Grammar: Sistem formal yang melengkapi aturan *grammar* dengan atribut dan aturan semantik untuk melakukan evaluasi nilai pada setiap node .

Visitor / Visit Functions: Pola desain di mana setiap jenis node memiliki fungsi `visit_<node>` yang bertugas:

1. Memanggil fungsi *visit* pada *child node* (traversal).
2. Mengakses atau memperbarui *Symbol Table*.
3. Menghitung atribut atau tipe semantik.
4. Menambahkan anotasi pada node .

BAB 2 : Perancangan & Implementasi

2.2. Implementasi Program

Program dibuat menggunakan bahasa C++ dan implementasi semantic analyser berada di file-file berikut :

1. File `include/SemanticAnalyzer.hpp` dan `src/SemanticAnalyzer.cpp`

File ini berfungsi sebagai komponen utama (*engine*) yang bertanggung jawab penuh atas penelusuran pohon sintaksis konkrit (CST), resolusi tipe data, serta penegakan aturan cakupan deklarasi (*scope checking*) dan validasi semantik bahasa Pascal. File ini mengisolasi seluruh logika bisnis analisis agar tidak bercampur dengan kontainer penyimpanan data.

2. File `include/SymbolTable.hpp` dan `src/SymbolTable.cpp`

File ini berfungsi sebagai struktur data pasif murni yang mengimplementasikan arsitektur tabel simbol standar Pascal-S (terdiri dari tabel `TAB`, `ATAB`, dan `BTAB`). Kelas ini mematuhi *Single Responsibility Principle* (SRP) dengan hanya menyediakan antarmuka (API) manajemen memori dan pencatatan tanpa mengetahui struktur pohon sintaksis maupun aturan tata bahasa.

2.1 Komponen Struktur Data Symbol Table

Berikut adalah implementasi *class*, *struct*, dan *enum* dari komponen tabel simbol pada berkas `include/SymbolTable.hpp`:

```
#pragma once
#include <string>
#include <vector>
#include <iostream>
#include "ASTNode.hpp"

class Node;      // Forward declaration murni untuk parameter
fungsi exportToFile
class ASTNode;   // Forward declaration murni untuk parameter
fungsi exportToFile

enum ObjectClass {
    OBJ_NONE = 0,
```

```

    OBJ_CONSTANT,
    OBJ_VARIABLE,
    OBJ_TYPE,
    OBJ_PROCEDURE,
    OBJ_FUNCTION
};

enum DataType {
    TYPE_NONE = 0,
    TYPE_INTEGER,
    TYPE_REAL,
    TYPE_CHAR,
    TYPE_BOOLEAN,
    TYPE_STRING,
    TYPE_ARRAY,
    TYPE_RECORD
};

struct TabEntry {
    std::string name;
    int link;
    ObjectClass obj;
    DataType type;
    int ref;
    int nrm;
    int lev;
    int adr;
};

struct AtabEntry {
    DataType xtyp;
    DataType etyp;
    int eref;
    int low;
    int high;
    int elsz;
    int size;
};

```

```

struct BtabEntry {
    int last;
    int lpar;
    int psze;
    int vsze;
};

class SymbolTable {
private:
    std::vector<TabEntry> tab;
    std::vector<AtabEntry> atab;
    std::vector<BtabEntry> btab;

    void initializePredefinedIdentifiers();

public:
    SymbolTable();
    ~SymbolTable() = default;

    // API Publik Manajemen Data TAB
    int addEntry(const std::string& name, ObjectClass obj, DataType
type, int ref, int nrm, int lev, int adr, int link = 0);
    int lookupIndex(const std::string& name) const;
    TabEntry getTabEntry(int index) const;
    size_t getTabSize() const { return tab.size(); }

    // API Publik Manajemen Data BTAB (Scope)
    int pushNewBlock(int lastIdx);
    void updateCurrentBlockLast(int lastIdx);
    void incrementCurrentBlockVsze(int lastIdx);
    void updateCurrentBlockLpar(int lparCount);
    void resetBlockZero();

    // API Publik Manajemen Data ATAB (Array)
    int buildAtabEntryDirect(DataType xtyp, DataType etyp, int
eref, int low, int high, int elsz, int size);

    // Fungsi Cetak Laporan
    void exportToFile(const std::string& filepath, Node* cstRoot,

```

```
ASTNode* astRoot) const;
    void printTab() const;
};
```

2.2 Komponen Logika Semantic Analyzer

Berikut adalah deklarasi kelas `SemanticAnalyzer` pada berkas `include/SemanticAnalyzer.hpp` yang menjembatani pengisian data dari abstrak sintaksis konkrit menuju kontainer tabel simbol:

```
#pragma once
#include "Node.hpp"
#include "SymbolTable.hpp"
#include <string>

class SemanticAnalyzer {
private:
    SymbolTable& symbolTable;
    int currentLev = 0;
    int currentBlock = 0;

    // Fungsi internal untuk pemrosesan navigasi pohon dan parsing
    data semantik
    void traverseNode(Node* node, int lev);
    DataType resolveTypeFromNode(Node* typeNode) const;
    std::string extractRawValue(const std::string& formattedLexeme)
const;
    int buildArrayEntry(Node* arrayTypeNode);

public:
    SemanticAnalyzer(SymbolTable& table);
    ~SemanticAnalyzer() = default;

    // Titik masuk utama untuk menganalisis pohon sintaksis konkrit
    (CST)
    void analyze(Node* cstRoot);
};
```


2.3. Aturan Kompatibilitas Tipe (Type Compatibility Rules)

Dalam implementasi *Semantic Analysis* pada bahasa Arion, dua tipe data dianggap kompatibel jika memenuhi salah satu kondisi berikut:

- Kesamaan Tipe: Kedua tipe memiliki tipe yang identik. Hal ini diperiksa melalui pencocokan nilai enum `DataType` yang sama (misalnya `leftType == rightType` pada evaluasi `BinOpNode`).
- Relasi Subrange: Salah satu tipe adalah subrange dari tipe lainnya (misalnya `[1..10]` dengan `Integer`), atau kedua tipe merupakan subrange yang berasal dari tipe dasar yang sama. Pada fungsi `resolveTypeFromNode`, tipe `RANGE` secara otomatis dikoreksi dan dikenali memiliki tipe dasar `TYPE_INTEGER`.
- Kesamaan Panjang String: Kedua tipe memiliki tipe `String` dan memiliki panjang karakter yang sama. Tipe string dasar akan dipetakan ke `TYPE_STRING`.

2.4. Aturan Kompatibilitas Penugasan (Assignment Compatibility Rules)

Sebuah tipe data `T2` dinyatakan *assignment-compatible* dengan tipe `T1` (sehingga penugasan `T1 := T2` diperbolehkan) jika memenuhi salah satu kondisi berikut pada fungsi `checkTypeCompatibility`:

- Promosi Real: `T1` bertipe `Real` dan `T2` bertipe `Integer`. Aturan ini diizinkan secara eksplisit pada blok kode ini

```
if (targetType != TYPE_NONE && valType != TYPE_NONE) {  
    if (targetType == TYPE_REAL && valType == TYPE_INTEGER) {  
    }  
    else if (targetType == TYPE_RECORD || valType == TYPE_RECORD ||  
            targetType == TYPE_ARRAY || valType == TYPE_ARRAY) {  
    }  
    else if (targetType != valType) {  
        reportError("Type mismatch dalam assignment. Tidak dapat menetapkan nilai ke target.");  
    }  
}
```

- Subset Nilai: `T1` dan `T2` kompatibel, keduanya bertipe `Integer`, `Boolean`, atau `Char`, dan nilai yang dimiliki `T2` adalah subset dari nilai yang dimiliki `T1` (misalnya penugasan nilai `5` ke tipe subrange `[1..10]`). Batas bawah dan atas akan divalidasi saat pembentukan array/range melalui fungsi `buildArrayEntry`.
- Kesamaan String: `T1` dan `T2` kompatibel dan keduanya bertipe `String` (`targetType == valType`` di mana keduanya bernilai `TYPE_STRING``).

2.5. Definisi Tipe Dasar (Simple Types)

Dalam implementasi lu, tipe dasar yang dikenali oleh *Semantic Analyzer* adalah sebagai berikut:

- Real: Tipe untuk nilai *realcon*. Dipetakan ke enum *TYPE_REAL*. Di dalam AST, NumberNode yang memiliki karakter titik (`.`) akan diidentifikasi sebagai tipe ini.
- Integer: Tipe untuk nilai *intcon*. Dipetakan ke enum *TYPE_INTEGER*. NumberNode tanpa komponen pecahan otomatis bertipe *TYPE_INTEGER*.
- Char: Tipe untuk nilai *charcon*. Dipetakan ke enum *TYPE_CHAR*.
- Boolean: Tipe untuk ekspresi yang dihasilkan dari operator *notsy* (NOT), *andsy* (AND), *orsy* (OR), serta *relational operators* (seperti *==*, *<*, *>*, dll). Di dalam fungsi *getExprType*, semua operator relasional dan logika tersebut diatur untuk mengembalikan *TYPE_BOOLEAN*.
- String: Tipe untuk nilai *string*. Dipetakan ke enum *TYPE_STRING* yang diekstrak dari *StringNode*.
- Subrange: Tipe yang batasannya ditentukan oleh konstanta. Diwakili oleh struktur node *RANGE*, di mana batas bawah (*lowBound*) dan batas atas (*HighBound*) diparsing menggunakan *std::stoi*.
- Enumerated: Tipe yang ditentukan oleh *identifier* yang dideklarasikan secara berurutan. Diwakili oleh tipe *ENUMERATED* yang dievaluasi menjadi tipe dasar ordinal (*TYPE_INTEGER*) pada tabel simbol.

1. Daftar Predefined Identifier

Sesuai dengan implementasi di *SymbolTable.cpp*, berikut adalah *predefined identifier* yang terdaftar dalam tabel simbol untuk mendukung tipe data dasar dan literasi logika

Identifier	Keterangan
<i>real</i>	Tipe data untuk nilai numerik pecahan (<i>floating point</i>).
<i>integer</i>	Tipe data untuk nilai numerik bulat.
<i>char</i>	Tipe data untuk karakter tunggal.
<i>boolean</i>	Tipe data untuk nilai logika (<i>true/false</i>).
<i>string</i>	Tipe data untuk urutan karakter.
<i>true</i>	Konstanta boolean dengan nilai benar.
<i>false</i>	Konstanta boolean dengan nilai salah.

2. Semantic Actions

Berikut adalah *Semantic Action* yang digunakan dalam proses konversi *Concrete Syntax Tree* (CST) menjadi *Abstract Syntax Tree* (AST) selama fase pembangunan AST di proyek kompilator ini

Production Rule	Semantic Rule
<code><assignment> -> ID := <expr></code>	<code>assign.node = new AssignNode(new VarNode(ID.lexeme), expr.node)</code>
<code><factor> -> INTCON</code>	<code>factor.node = new NumberNode(INTCON.value)</code>
<code><factor> -> REALCON</code>	<code>factor.node = new NumberNode(REALCON.value)</code>
<code><factor> -> STRING</code>	<code>factor.node = new StringNode(STRING.value)</code>
<code><factor> -> ID</code>	<code>factor.node = new VariableNode(ID.lexeme)</code>
<code><simple-expr> -> <term> + <term></code>	<code>expr.node = new BinOpNode('+', term1.node, term2.node)</code>
<code><simple-expr> -> <term> - <term></code>	<code>expr.node = new BinOpNode('-', term1.node, term2.node)</code>
<code><term> -> <factor> * <factor></code>	<code>term.node = new BinOpNode('*', factor1.node, factor2.node)</code>
<code><term> -> <factor> div <factor></code>	<code>term.node = new BinOpNode('div', factor1.node, factor2.node)</code>
<code><if-stmt> -> IF <expr> THEN <stmt></code>	<code>if.node = new IfNode(expr.node, then.node)</code>

BAB 3 : Pengujian

Input	Output
Case 1	
<pre> program Hello; var a, b, i: integer; function AddTen(x: integer): integer; begin AddTen := x + 10; end; begin a := 5; b := AddTen(a); writeln('Result = ', b); end. </pre>	<pre> --- SYMBOL TABLE (TAB) --- idx id obj type ref nrm lev adr link ----- 33 Hello 4 0 0 1 0 0 0 34 a 2 1 0 1 0 0 0 35 b 2 1 0 1 0 0 0 36 i 2 1 0 1 0 0 0 37 AddTen 5 1 0 1 0 0 0 38 x 2 1 0 1 1 0 0 --- ARRAY TABLE (ATAB) --- idx xtyp etyp eref low high elsz size ----- --- BLOCK TABLE (BTAB) --- idx last lpar psze vsze ----- 0 0 0 0 0 1 37 0 0 3 2 38 1 0 1 --- DECORATED AST --- └─ BlockNode └─ BlockNode └─ VarDeclNode(Type: integer) ├── a ├── comma ├── b ├── comma └─ i └─ SubprogramDecl(AddTen) └─ [NULL BLOCK] └─ BlockNode └─ BlockNode ├── AssignNode │ ├── VariableNode(a) │ └─ NumberNode(5) ├── AssignNode │ ├── VariableNode(b) │ └─ CallNode(func: 'AddTen') │ └─ VariableNode(a) └─ CallNode(func: 'writeln') ├── StringNode('Result = ') └─ VariableNode(b) </pre>

Case 2

```
program CommentTest;
{ Ini adalah komentar satu baris }
(* Ini adalah komentar
   multi-baris Pascal klasik *)
begin
  writeln('Testing quotes: it's working');
  { Komentar di akhir program }
end.
```

```
--- SYMBOL TABLE (TAB) ---
idx  id          obj          type      ref  nrm  lev  adr  link
-----
33   CommentTest  4              0        0    1    0    0    0

--- ARRAY TABLE (ATAB) ---
idx  xtyp  etyp  eref  low  high  elsz  size
-----

--- BLOCK TABLE (BTAB) ---
idx  last  lpar  psze  vsze
-----
0     0     0     0     0
1    33     0     0     0

--- DECORATED AST ---
└─ BlockNode
   └─ BlockNode
      └─ BlockNode
         └─ CallNode(func: 'writeln')
            └─ StringNode('Testing quotes: it's working')
```

Case 3

```
program ErrorTest;
var
  a: integer;
  b: real;
  a: string; {SUPAYA ERROR }
begin
  a := 10;
end.
```

```
Semantic Error: Variabel 'a' sudah dideklarasikan pada scope ini.

Analisis semantik gagal
```

Case 4

```

program ArrayTest;
const
  PI = 3.14;
var
  numbers: array[1..5] of real;
  i: integer;
begin
  for i := 5 downto 1 do
    begin
      numbers[i] := 1 * PI;
    end;
  end;
end.
program RecordTest;
type
  Point = record
    x, y: integer;
  end;
var
  p1: Point;
begin
  p1.x := 10;
  p1.y := 20;

  { Menguji apakah lexer bingung antara . (period) dan .. (range) }
  writeln('Coordinates: ', p1.x, ', ', p1.y);
end.

```

```

--- SYMBOL TABLE (TAB) ---
idx  id      obj      type      ref  nrm  lev  adr  link
-----
33   ArrayTest  4          0          0    1    0    0    0
34    PI        1          2          0    1    0    0    0
35   numbers    2          6          1    1    0    0    0
36    i         2          1          0    1    0    0    0

```

```

--- ARRAY TABLE (ATAB) ---
idx  xtyp  etyp  eref  low  high  elsz  size
-----
1     1     2     0     0    0     1     1

```

```

--- BLOCK TABLE (BTAB) ---
idx  last  lpar  psze  vsze
-----
0     0     0     0     0
1    36     0     0     2

```

```

--- DECORATED AST ---
└─ BlockNode
   └─ BlockNode
      └─ VarDeclNode(Type: arraysy)
         └─ numbers
      └─ BlockNode
         └─ BlockNode
            └─ ForNode(var: 1)
               └─ NumberNode(5)
               └─ NumberNode(1)
               └─ BlockNode
                  └─ BlockNode
                     └─ AssignNode
                        └─ ArrayAccessNode(array: numbers)
                           └─ VariableNode(i)
                           └─ BinOpNode(op: '*')
                              └─ VariableNode(i)
                              └─ VariableNode(PI)

```

```
const
  MaxItems = 100;
  PiValue = 3.14159;
  AppName = 'FarisCompiler';

type
  TIndex = 1..100;
  TColor = (Red, Green, Blue);

  TPoint = record
    X, Y: real;
    Color: TColor;
  end;

  TPath = array[1..50] of TPoint;
  TValues = array[1..10] of integer;

var
  GlobalPath: TPath;
  Grid: TValues;
  CurrentColor: TColor;
  i: integer;
  nextIdx: integer;
  TotalDistance: real;

procedure InitGrid(StartVal: integer);
var
  r: integer;
begin
  for r := 1 to 10 do
    Grid[r] := StartVal * r;
  end;
end;

function CalculateDist(P1, P2: TPoint): real;
var
  dx, dy: real;
begin
  dx := P1.X - P2.X;
  dy := P1.Y - P2.Y;
  CalculateDist := (dx * dx) + (dy * dy);
end;

begin
  { 1. Inisialisasi Record di dalam Array }
  for i := 1 to 50 do
    begin
      GlobalPath[i].X := i * 1.5;
      GlobalPath[i].Y := i * 2.5;

      if (i MOD 2) == 0 then
        GlobalPath[i].Color := Red
      else
        GlobalPath[i].Color := Blue;
      end;
    end;

  { 2. Pemanggilan Procedure dengan Parameter Aktual }
  InitGrid(5);

  { 3. Pemanggilan Function di dalam Ekspresi Matematika }
  TotalDistance := 0.0;
  i := 1;
  while i < 50 do
    begin
      nextIdx := i + 1;
      TotalDistance := TotalDistance + CalculateDist(GlobalPath[i], GlobalPath[nextIdx]);
      i := i + 1;
    end;

  { 4. Evaluasi Case Statement }
  CurrentColor := GlobalPath[10].Color;
  case CurrentColor of
    Red, Blue:
      begin
        TotalDistance := TotalDistance * PiValue;
      end;
    Green:
      TotalDistance := 0.0;
  end;

  { 5. Repeat Until dengan Operator Divisi dan Relasional }
  i := 10;
  repeat
    Grid[i] := Grid[i] div 2;
    i := i - 1;
  until i == 0;

end.
```

--- SYMBOL TABLE (TAB) ---

idx	id	obj	type	ref	nrm	lev	adr	link
33	AdvancedCompilerTest4		0	0	1	0	0	0
34	MaxItems	1	1	0	1	0	0	0
35	PiValue	1	2	0	1	0	0	0
36	AppName	1	0	0	1	0	0	0
37	TIndex	3	1	0	1	0	0	0
38	TColor	3	1	0	1	0	0	0
39	TPoint	3	7	0	1	0	0	0
40	TPath	3	6	1	1	0	0	0
41	TValues	3	6	2	1	0	0	0
42	GlobalPath	2	6	1	1	0	0	0
43	Grid	2	6	2	1	0	0	0
44	CurrentColor	2	1	0	1	0	0	0
45	i	2	1	0	1	0	0	0
46	nextIdx	2	1	0	1	0	0	0
47	TotalDistance	2	2	0	1	0	0	0
48	InitGrid	4	0	0	1	0	0	0
49	StartVal	2	1	0	1	1	0	0
50	r	2	1	0	1	1	0	0
51	CalculateDist	5	2	0	1	0	0	0
52	P1	2	7	0	1	1	0	0
53	P2	2	7	0	1	1	0	0
54	dx	2	2	0	1	1	0	0
55	dy	2	2	0	1	1	0	0

--- ARRAY TABLE (ATAB) ---

idx	xtyp	etyp	eref	low	high	elsz	size
1	1	7	0	0	0	1	1
2	1	1	0	0	0	1	1

--- BLOCK TABLE (BTAB) ---

idx	last	lpar	psze	vsze
0	0	0	0	0
1	48	0	0	6
2	51	1	0	2
3	55	2	0	4

--- DECORATED AST ---

```
└─ BlockNode
  └─ BlockNode
    └─ VarDeclNode(Type: TPath)
      └─ GlobalPath
    └─ SubprogramDecl(InitGrid)
      └─ [NULL BLOCK]
    └─ SubprogramDecl(CalculateDist)
      └─ [NULL BLOCK]
  └─ BlockNode
    └─ BlockNode
      └─ ForNode(var: i)
        └─ NumberNode(1)
        └─ NumberNode(50)
        └─ BlockNode
          └─ BlockNode
            └─ AssignNode
              └─ FieldAccessNode(GlobalPath.X)
              └─ BinOpNode(op: '*')
                └─ VariableNode(i)
                └─ NumberNode(1.5)
            └─ AssignNode
              └─ FieldAccessNode(GlobalPath.Y)
              └─ BinOpNode(op: '*')
                └─ VariableNode(i)
                └─ NumberNode(2.5)
            └─ IfNode
              └─ Condition:
                └─ BinOpNode(op: '==')
                  └─ BinOpNode(op: 'MOD')
                    └─ VariableNode(i)
                    └─ NumberNode(2)
                    └─ NumberNode(0)
              └─ Then:
                └─ AssignNode
                  └─ FieldAccessNode(GlobalPath.Color)
                  └─ VariableNode(Red)
              └─ Else:
                └─ AssignNode
                  └─ FieldAccessNode(GlobalPath.Color)
                  └─ VariableNode(Blue)
          └─ CallNode(func: 'InitGrid')
            └─ NumberNode(5)
        └─ AssignNode
          └─ VariableNode(TotalDistance)
          └─ NumberNode(0.0)
        └─ AssignNode
          └─ VariableNode(i)
          └─ NumberNode(1)
```



```

└─ BlockNode
  └─ BlockNode
    └─ AssignNode
      └─ VariableNode(1)
      └─ NumberNode(1)
    └─ WhileNode
      └─ BinOpNode(op: '<')
      └─ VariableNode(1)
      └─ NumberNode(50)
      └─ BlockNode
        └─ BlockNode
          └─ AssignNode
            └─ VariableNode(nextIdx)
            └─ BinOpNode(op: '+')
              └─ VariableNode(1)
              └─ NumberNode(1)
          └─ AssignNode
            └─ VariableNode(TotalDistance)
            └─ BinOpNode(op: '+')
              └─ VariableNode(TotalDistance)
              └─ CallNode(func: 'CalculateDist')
                └─ ArrayAccessNode(array: GlobalPath)
                  └─ VariableNode(1)
                └─ ArrayAccessNode(array: GlobalPath)
                  └─ VariableNode(nextIdx)
          └─ AssignNode
            └─ VariableNode(1)
            └─ BinOpNode(op: '+')
              └─ VariableNode(1)
              └─ NumberNode(1)
        └─ AssignNode
          └─ VariableNode(CurrentColor)
          └─ FieldAccessNode(GlobalPath.Color)
        └─ CaseNode
          └─ VariableNode(CurrentColor)
          └─ Branch: Green
            └─ AssignNode
              └─ VariableNode(TotalDistance)
              └─ NumberNode(0.0)
        └─ AssignNode
          └─ VariableNode(1)
          └─ NumberNode(10)
        └─ RepeatNode
          └─ BlockNode
            └─ AssignNode
              └─ ArrayAccessNode(array: Grid)
                └─ VariableNode(1)
              └─ BinOpNode(op: '/')
                └─ ArrayAccessNode(array: Grid)

```

BAB 4 : Kesimpulan dan Saran

4.1. Kesimpulan

Berdasarkan hasil perancangan dan implementasi pada fase *Semantic Analysis*, dapat ditarik kesimpulan sebagai berikut:

1. Algoritma *semantic analysis* berhasil diterapkan dengan menelusuri CST yang dihasilkan parser untuk membentuk informasi semantik program. Proses ini mencakup pencatatan identifier ke dalam *symbol table*, pengenalan tipe data, serta pembentukan keluaran berupa *decorated AST* dan tabel simbol.
2. Implementasi *symbol table* berhasil digunakan untuk menyimpan informasi penting seperti nama identifier, jenis objek, tipe data, level scope, referensi array, dan informasi blok. Dengan adanya struktur TAB, ATAB, dan BTAB, program dapat merepresentasikan deklarasi variabel, konstanta, tipe, prosedur, dan fungsi secara lebih teratur.
3. Program telah dilengkapi dengan penanganan beberapa kesalahan semantik dasar, seperti deklarasi variabel ganda, dengan menampilkan pesan kesalahan yang informatif.

4.2. Saran

Untuk pengembangan lebih lanjut dari modul *Semantic Analysis* proyek ini, disarankan beberapa pembaruan teknis berikut:

1. Implementasi Type Checking Komprehensif: Menambahkan mekanisme pengecekan tipe data lintas node pada Abstract Syntax Tree (AST). Hal ini diperlukan untuk memvalidasi operasi matematika, mencegah *type mismatch* (misalnya penugasan *string* ke variabel *integer*), dan memastikan kesesuaian jumlah serta tipe argumen pada pemanggilan subprogram (*Procedure/Function Call*).
2. Perluasan Deteksi Semantic Error: Mengembangkan jangkauan *error handling* semantik untuk mendeteksi anomali operasional, seperti penggunaan *undeclared identifier* (variabel/fungsi yang dipanggil tanpa deklarasi), pemanggilan *field record* yang tidak eksis, serta pelanggaran batas akses indeks array (*array out of bounds*).
3. Optimasi Abstract Syntax Tree (AST): Menerapkan teknik *Constant Folding* pada fase pembangunan AST. Operasi matematika yang melibatkan konstanta literal statis (misalnya $10 * 2$) dapat dievaluasi secara langsung di level kompilator menjadi 20 untuk mereduksi beban instruksi sebelum masuk ke fase *Code Generation*.
4. Persiapan Code Generation: Mengkalkulasi dan merekam ukuran alokasi memori aktual secara lebih presisi (menggunakan byte/word offset) di dalam Symbol Table. Data ini krusial sebagai fondasi mutlak untuk pembentukan *Intermediate Representation* (IR) atau bahasa mesin (Assembly) pada milestone kompilator berikutnya.

Lampiran

- Link Repository Github = <https://github.com/kalkabena/XYZ-Tubes-IF2224-2026.git>
- Tabel Pembagian Tugas:

NIM	Nama	Pembagian Tugas	Persentase Kontribusi
13524114	Mirza Tsabita Wafa'ana	Mengerjakan bagian AST_Tree dan mengerjakan laporan	25%
13524119	Nathanael Shane Bennet	Mengerjakan bagian symbol table dan mengerjakan laporan	25%
13524130	Faris Wirakusuma Triawan	Mengerjakan bagian symbol table dan mengerjakan laporan	25%
13524144	Jonathan Harijadi	Mengerjakan bagian AST_Tree dan mengerjakan laporan	25%

Referensi

Free Software Foundation. (n.d.). *Bison - The Yacc-compatible Parser Generator*. Diakses dari <https://www.gnu.org/software/bison/manual/>.

Louden, K. C. (1997). *Compiler Construction: Principles and Practice*. PWS Publishing Company.